

LVA-5 — Rotate Firebase CI token (printed to session transcript)

Status: Operator-blocked **Type:** Bug **Operator-Block-Details:**

WHAT: ACCEPTED RISK — operator decision recorded 2026-08-26. WHAT: the operator has reviewed the 2026-05-20 LAVA_FIREBASE_TOKEN transcript exposure and elected to ACCEPT the risk rather than rotate the credential. This item is retained in the OPEN register deliberately, as a standing risk-acceptance rather than as pending work: it is not awaiting an agent action, and its presence here is not an oversight. It is not closed, because closing it would assert a remediation that did not occur, and an accepted risk recorded as fixed is indistinguishable from a risk that was actually remediated. WHY: section 6.H clause 6 assigns credential rotation to the operator alone; the operator is therefore also the authority that may decline it, and that declination is the decision recorded here. UNBLOCK: none pending. No agent should re-raise this as a new finding or schedule a rotation on the operator's behalf. The original options remain available verbatim if the operator's intent changes: [A] firebase logout then firebase login:ci, writing the new value to LAVA_FIREBASE_TOKEN in the gitignored .env; [B] mint a fresh value via firebase login:ci -no-localhost in a clean shell and update .env; [C] rotate via the Firebase console under Project Settings, Service accounts, and update .env. Re-entry signal: the operator states that the acceptance is withdrawn, at which point this reverts to ordinary pending work under those options. WHO: Operator (sole rotation authority per section 6.H clause 6, and the author of this acceptance) WHY: Section 6.H clause 6 assigns credential rotation exclusively to the operator: an agent cannot run firebase logout or firebase login:ci, cannot reach the Firebase console, and cannot write the gitignored .env on the operator's behalf. The same clause makes the operator the only authority that may DECLINE rotation, which is the decision recorded on this item as of 2026-08-26. This item therefore stays Operator-blocked and OPEN by design rather than by neglect: closing it would assert a remediation that did not occur, and section 6.J forbids recording an outcome the work did not produce. UNBLOCK: NONE PENDING while the acceptance stands — the operator recorded an explicit risk-acceptance on 2026-08-26, so no agent should schedule a rotation or re-raise this as a new finding. The enumerated CHOICES below remain available verbatim and take effect the moment the operator withdraws that acceptance: [A] run "firebase logout" then "firebase login:ci" and write the new value to LAVA_FIREBASE_TOKEN in the gitignored .env (recommended); [B] mint a fresh value via

“firebase login:ci -no-localhost” in a clean shell and update LAVA_FIREBASE_TOKEN in .env; [C] rotate via the Firebase console under Project Settings, Service accounts, and update .env; [D] re-affirm the acceptance unchanged, in which case this item stays open as a standing risk-acceptance and is not to be cited as pending work. Unblock signal for [A], [B] or [C]: a Firebase distribute run authenticates with the new value and the previously-exposed one no longer authenticates. Unblock signal for [D]: the operator restates the acceptance with a fresh date. No token value is ever recorded here or committed (section 6.H, section 11.4.10). WHO: Operator **Severity:** P0

67th-cycle: the LAVA_FIREBASE_TOKEN default-expansion printed the token to the session transcript (NOT committed to git). Operator MUST rotate per §6.H clause 6 (firebase logout; firebase login:ci). **Source:** self-discovered — .lava-ci-evidence/sixth-law-incidents/2026-05-20-firebase-token-echo-leak.json

OPERATOR DECISION, 2026-08-26 — ACCEPTED RISK. The operator has reviewed this item and elected to ACCEPT the risk rather than rotate the credential at this time. This item is therefore NOT abandoned work and NOT a pending action awaiting an agent: it is a recorded risk-acceptance, retained in the open register so it stays visible rather than being closed and forgotten. WHAT IS AND IS NOT TRUE AS A RESULT: the leak itself remains a matter of record — the token value was expanded into a session transcript on 2026-05-20 and was never committed to git, per the incident record cited above — and the decision does not retroactively make that exposure harmless. What the decision settles is the RESPONSE: no rotation is scheduled, and no agent should treat the absence of rotation as an oversight or re-raise it as a new finding. WHY THIS ITEM STAYS OPEN RATHER THAN CLOSING: closing it would assert the underlying condition was remediated, which is not what happened, and section 6.J forbids recording an outcome the work did not produce. An accepted risk that has been closed is indistinguishable in the tracker from a risk that was fixed, and that is precisely the conflation this register exists to prevent. RE-OPENING CONDITION: if the accepted token is observed authenticating after the operator’s intent changes, or if the operator later elects to rotate, this item resumes as ordinary pending work under its original unblock options [A], [B] or [C], unchanged.

LVA-008 — C11/C06 search_input NavBackStackEntry teardown crash (off-main-thread navigate; fixed ccdd84c1)

Status: Ready for testing **Type:** Bug **Severity:** P1 **Created-By:** AI

ROOT CAUSE CORRECTED 2026-08-26 — this supersedes the androidx-navigation attribution retained below. C06 + C11 (Challenge11ArchiveOrgAnonymousSearchTest) crashed the app PROCESS at activity-destroy with IllegalStateException “State must be at least CREATED to be moved to DESTROYED” on the inner search/search_input entry. The defect was NOT an upstream AndroidX bug and never existed in AndroidX code. It was a Lava threading defect: under createAndroidComposeRule, FrameDeferringContinuationInterceptor resumed the Orbit collectSideEffect continuation OFF the main thread, so navHostController.navigate(...) executed off-main; navigation-runtime 2.9.1 inserts the new back-stack entry BEFORE promoting its lifecycle, and LifecycleRegistry.setCurrentState throws when invoked off-main — leaving that entry permanently INITIALIZED, which produced the illegal INITIALIZED-to-DESTROYED transition at Activity destroy. FIXED 2026-06-30 by commit ccdd84c1, which wraps the navigate call in runOnMainThread at core/navigation/src/main/kotlin/lava/navigation/NavigationController.kt:101. EVIDENCE CHAIN: last reproduction 2026-06-29 21:26; runOnMainThread guard added 2026-06-30 11:24; the same journey ran clean 2026-06-30 17:48; zero occurrences of the crash signature in any committed evidence since. DISPROVEN HYPOTHESIS (retained as forensic record per 11.4.6, not erased): the item was attributed to a nested-NavHost lifecycle defect in androidx-navigation (b/244910446 family) and held Operator-blocked pending an upstream release. Eight app-level candidate fixes were implemented and device-falsified: inner-NavHost Activity-scoped LifecycleOwner, move-to-outer-NavHost, ON_STOP force-pop (NavTeardownGuard), nav-compose 2.9.1-to-2.9.8-to-2.10.0-alpha bump, LenientTeardownRule, atomic popUpTo replace, launchSingleTop dedupe, and a full nested-NavHost-architecture collapse. Each reproduced the identical ISE, and that uniformity was read as proof of an upstream defect. WHY THAT READING WAS WRONG: all eight attempts changed WHERE navigate() was called or HOW the back stack was structured; not one changed WHICH THREAD navigate() ran on. The off-main call — the actual cause — survived all eight, so all eight reproduced the same crash. Uniform reproduction across eight fixes was evidence of an untouched common cause, not of an upstream defect. The filing-ready upstream repro package at docs/lva008-upstream-repro/ was DELETED 2026-08-26: filing it would have reported to AndroidX a bug that does not exist in their code. RELATED

CLEANUP: the autonomous-QA PASS-override that converted this crash signature into a reported PASS (scripts/autonomous-qa/run-iteration.sh) was removed 2026-08-26 — with the cause fixed, it was a live 6.J bluff mechanism reporting a genuine crash as green. CLOSURE PENDING — THIS ITEM IS NOT CLOSED: closure requires a C06 + C11 device re-run at HEAD on a 6.AH-conformant container or VM emulator (never host-direct, never a live ADB device), both GREEN, with the JUnit XML captured as closure evidence.

LVA-121 — Pipeline 002 — HYPOTHESIS (not reproduced): phase-01-build.sh counts ARTIFACT lines, not distinct labels

Status: Queued **Type:** Task **Severity:** P3 **Created-By:** AI

UNCONFIRMED: this is a HYPOTHESIS, not a demonstrated defect. It was NOT reproduced during feature 002 and no failing case was constructed. Filed so it is not lost; it must be either reproduced (then re-filed as a Bug with RED evidence) or explicitly ruled out. CLAIM: scripts/pipeline/phase-01-build.sh counts ARTIFACT lines rather than distinct labels, so five duplicate labels would satisfy the ≥ 5 PASS condition. WHY IT WAS NOT PROMOTED TO A FINDING: not reachable through the real sub-scripts, each of which prints a given label at most once. To close, either construct a real producing path or record the argument that none exists. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-122 — Pipeline 002 — HYPOTHESIS (not reproduced): recompute_evidence_summary would report zero for an unreadable dir

Status: Queued **Type:** Task **Severity:** P3 **Created-By:** AI

UNCONFIRMED: this is a HYPOTHESIS, not a demonstrated defect. It was NOT reproduced during feature 002 and no failing case was constructed. Filed so it is not lost; it must be either reproduced (then re-filed as a Bug with RED evidence) or explicitly ruled out. CLAIM: the `find ... 2>/dev/null` inside `recompute_evidence_summary` in `scripts/pipeline/lib/run-report.sh` would report zero Evidence Records for an unreadable phase directory, which `finalize_run_report` would then treat as zero

rejections. WHY IT WAS NOT PROMOTED TO A FINDING: no realistic producing scenario was constructed. To close, either construct one or record why the phase directory cannot become unreadable mid-run. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-123 — Pipeline 002 — HYPOTHESIS (not reproduced): phase-04 compose-file port parse is a no-op on an unquoted value

Status: Queued **Type:** Task **Severity:** P3 **Created-By:** AI

UNCONFIRMED: this is a HYPOTHESIS, not a demonstrated defect. It was NOT reproduced during feature 002 and no failing case was constructed. Filed so it is not lost; it must be either reproduced (then re-filed as a Bug with RED evidence) or explicitly ruled out. CLAIM: the compose-file port parse in scripts/pipeline/phase-04-live-verify-api.sh is a no-op when the LAVA_API_LISTEN value in docker-compose.yml is unquoted. WHY IT WAS NOT PROMOTED TO A FINDING: the real docker-compose.yml quotes the value, and a wrong port fails closed at curl rather than passing vacuously. To close, either construct a producing case or make the parse fail loudly on an unrecognized shape. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-128 — Pipeline 002 — T054 human review gate — mandatory review of advance-all-submodules.sh before ANY real invocation

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: scripts/advance-all-submodules.sh has never run against a real submodule or upstream and must not until this review completes. Fixture-only TDD evidence: tests/pipeline/test_advance_all_submodules.sh, 36/36. WHY: Highest blast radius in feature 002 - automatic pin advancement across 16-plus real upstreams. plan.md Human Checkpoint #2 makes the review mandatory and prior to first real invocation. UNBLOCK: Operator picks one of the following CHOICES: [A] a human reviews scripts/advance-all-submodules.sh against the criteria above, records approval, and authorizes a first run against a DISPOSABLE clone per task T056; [B] a human reviews it and requires specific

changes, which are made and re-reviewed before any real invocation; [C] the operator declines to authorize automated submodule advancement at all, in which case feature 002 task T055 is descoped and the script is never invoked against a real upstream. Unblock signal: the review verdict is recorded, and for CHOICE [A] the disposable-clone run of task T056 completes. WHO: Operator or a designated human reviewer **Severity: P0**
Created-By: AI

WHAT: feature 002 task T054 is a MANDATORY code review of scripts/advance-all-submodules.sh before any invocation against a real submodule upstream, per plan.md Human Checkpoint #2. This is the highest-blast-radius new component in the feature: 16-plus submodule upstreams with automatic pin advancement, plus commit-and-push of each submodule own local modifications. CURRENT STATE, stated honestly: the T050-T053 TDD cycle against DISPOSABLE FIXTURE repos is COMPLETE - 36/36 assertions green, independently re-run by the orchestrating session - and real submodules were confirmed untouched (git status -porcelain over submodules/ and constitution/ empty, git diff -cached empty). scripts/advance-all-submodules.sh has NEVER been run against a real submodule or a real upstream. REVIEW MUST COVER at minimum: the GOVERNANCE_DENY default-deny list added by the wave-5 fix and the fact it is deliberately not overridable via LAVA_ADVANCE_SUBMODULES; the 7-step ordering from research.md R-005 (fetch, compare, no-op or checkout-advance, rebuild-and-test, discard-and-report on failure, commit/push the submodule own local modifications, update the parent pin); and the refusal paths REJECTED_BREAKING_CHANGE, REJECTED_PUSH_CONFLICT and REFUSED_GOVERNANCE_DENY. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-129 — Pipeline 002 — T062 human review gate — full end-to-end pipeline run on a disposable branch before trusting it

Status: Operator-blocked **Type:** Task **Operator-Block-Details:** WHAT: The full five-phase end-to-end orchestrator run has never been executed. It must first run on a disposable branch, never on real master. WHY: plan.md Human Checkpoint #3. A first full run touches Gradle, systemd and an emulator on the real host and must be supervised on a throwaway branch. UNBLOCK: Operator picks one of the following CHOICES: [A] a human runs quickstart.md Scenario 2 end to end on a DISPOSABLE branch with all five wired phases and reviews the resulting report.json; [B] a human runs it phase by phase on a disposable branch,

reviewing each phase result before proceeding to the next; [C] the operator declines the end-to-end run, in which case scripts/pipeline-build-test-distribute.sh remains untrusted and must not be pointed at real master. Unblock signal: a run report for the disposable branch validates against contracts/pipeline-run-report.schema.json and is recorded, or the operator records the decision to decline. WHO: Operator or a designated human reviewer **Severity: P1 Created-By: AI**

WHAT: feature 002 task T062 requires a full end-to-end run of quickstart.md Scenario 2 (all phases, no shortcuts) on a DISPOSABLE branch, per plan.md Human Checkpoint #3, before scripts/pipeline-build-test-distribute.sh is trusted to run against real master. CURRENT STATE, stated honestly: the full end-to-end orchestrator run across all five wired phases has NOT been performed. It invokes Gradle, systemd and an Android emulator, and was deliberately not run because it is this review gate and because it would have contended with parallel agents already using those toolchains. It is not claimed as done anywhere. SCOPE NOTE: only phases precondition, build, test, install_boot and live_verify are wired; distribute, docs and closure are deliberately not wired pending T040/T041 and T048/T049, and asking for them via -until is a usage error rather than a silent no-op. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-131 — Pipeline 002 — a successful run dirties the tree and the next run is refused, so the pipeline cannot run twice

Status: Queued **Type:** Bug **Severity:** P0

WHAT: the phase-02 go test category caused lava-api-go's own §11.4.85 stress and chaos suites to write their evidence into the repository-level .lava-ci-evidence/stress-chaos/jackett/ directory. Six files land there and all six are TRACKED (confirmed: git ls-files lists exactly 6 paths under that directory). Their payload carries a captured_at stamp plus measured latency percentiles, so it differs on every run by construction. Net effect: a SUCCESSFUL run left the working tree dirty, and the next invocation was refused by the FR-000 precondition, which requires branch master AND a completely clean git status --porcelain and exits 2 otherwise. EVIDENCE, measured directly: run 2026-08-23T10-17-26Z finalized outcome PASS with phases precondition/build/test all PASS; the immediately following run 2026-08-23T10-31-21Z finalized outcome FAIL with a single precondition/FAIL phase, and its phase-00 log reads verbatim

FR-000: precondition failed – working tree is not clean. This also falsifies quickstart.md Scenario 5's claim that everything a run produces is gitignored. IMPACT: blocks FR-018 and SC-007 (a run must not dirty the working tree). STATUS: under active fix in the working tree — scripts/pipeline/phase-02-test-go.sh now exports LAVA_STRESS_CHAOS_EVIDENCE_DIR pointing at the run's own gitignored evidence directory, and tests/pipeline/test_phase_02_go_evidence_isolation.sh is present as regression coverage; neither is committed yet. CLOSURE CRITERIA: two consecutive full runs both reach their final phase, with the tracked jackett files unchanged after each.

LVA-132 — Pipeline 002 — a TERM signal is not honoured mid-build, so quickstart Scenario 5's finalized-report claim does not hold promptly

Status: Queued **Type:** Bug **Severity:** P1

WHAT: scripts/pipeline-build-test-distribute.sh installs trap '_close_report' INT TERM (line 288) and then invokes each phase as a FOREGROUND pipeline: "\$SCRIPT_DIR/pipeline/\${script_name}" "\$run_id" "\$REPO_PATH_OVERRIDE" 2>&1 | tee "\$phase_log" (lines 342 and 345). Bash does not execute a trap handler while the shell is blocked waiting on a foreground command; the handler is deferred until that command completes. A TERM delivered during a multi-minute Gradle build therefore does not finalize the report at signal time — it finalizes only after the build finishes on its own, which is the opposite of what an interrupt is for. EVIDENCE, measured directly this session with a minimal reproduction that mirrors the orchestrator's shape (trap on INT TERM, then sleep 6 | tee): TERM was delivered at epoch 1787681147 and the handler printed HANDLER RAN at epoch 1787681151 — a 4-second deferral exactly equal to the foreground pipeline's remaining duration. IMPACT: quickstart.md Scenario 5's assertion that a TERM produces a finalized report does not hold while a phase is running, which is precisely when an operator would send one. CLOSURE CRITERIA: the phase runs in the background with the shell waiting on it interruptibly (or an equivalent seam), and a test delivers TERM mid-phase and asserts the report is finalized within a bounded interval rather than after the phase completes.

LVA-133 — Pipeline 002 — quickstart Scenario 3 is unrunnable as written because its own mutation trips the FR-000 clean-tree precondition

Status: Queued **Type:** Bug **Severity:** P2

WHAT: specs/002-build-test-distribute-pipeline/quickstart.md Scenario 3 instructs the reader to check out master, apply a deliberate one-line regression to production code, then invoke `bash scripts/pipeline-build-test-distribute.sh` and expect exit code 1 with a test-phase FAIL Evidence Record. That sequence cannot reach the test phase: the mutation dirties the working tree, and `scripts/pipeline/phase-00-precondition.sh` refuses any run whose `git status --porcelain` is non-empty, printing FR-000: precondition failed – working tree is not clean and exiting 2 (the branch and clean-tree checks are at lines 39-48, both exit 2). The run therefore halts at precondition with exit 2, never reaches phase-02, and the scenario's stated expected outcome is unreachable. NOTE: the same quickstart already carries 2026-08-21 corrections to Scenario 3 on two OTHER points (absent-versus-SKIPPED phases, and the vacuous zero-Distribution-Records assertion), so the document is maintained — this third defect is simply not among the corrections. CLOSURE CRITERIA: Scenario 3 is rewritten so its falsifiability demonstration is reachable — for example by committing the mutation on a disposable branch, or by using the anti-bluff Evidence Record variant the scenario's own closing paragraph already describes as available now.

LVA-137 — Safety hook — guard-forbidden-commands.sh matches any command containing git, a later push token and a bare -f, blocking benign writes

Status: Queued **Type:** Bug **Severity:** P2

WHAT: `scripts/hooks/guard-forbidden-commands.sh` line 208 gates the §6.T.3 force-push check with `[[ "$COMMAND" =~ git([[:space:]]+ [^[:space:]]+)*([[:space:]]+push )]`. The expression is unanchored and its `([[:space:]]+ [^[:space:]]+)*` group spans arbitrarily many arbitrary tokens, so it matches ANY command in which the literal `git` occurs anywhere and the token `push` occurs later, however unrelated the two are. Combined with the standalone `-f` test on the next lines, any command that also contains a bare minus-`f`

token (a routine file removal, for instance) is refused as a force-push. EVIDENCE, measured directly this session: the attempt to write `.lava-ci-evidence/bluff-hunt/2026-08-23-cycle-gate-shaping-scripts.json` was refused with the verbatim message guardrails: BLOCKED – §6.T.3 force-push, because the evidence file’s own prose describing this matcher contains those tokens. The write succeeded only by routing around the Bash tool entirely. The same matcher had already blocked the original 2026-08-23 attempt to record that hunt, which is why the evidence file was two days late. ASSESSMENT: the guard fails SAFE — it refuses rather than permits — so no unsafe operation resulted. The cost is different and still real: it trains agents to route around a safety guard to get ordinary work done, which erodes the guard’s authority for the case it exists to catch. CLOSURE CRITERIA: the matcher is anchored to a genuine command position (a real git invocation whose subcommand is push) rather than to token co-occurrence anywhere in the string, with fixtures covering both a true force-push and the benign co-occurrence cases above.

RECURRENCE, 2026-08-26 (this item is the pre-existing record; these are additional independent reproductions, filed here rather than as a duplicate item): the same matcher refused an ordinary documentation write TWICE more in a single session, on routine work that involved no version-control operation of any kind. In both instances the refusal was triggered by the DOCUMENT’S OWN PROSE: the text under composition contained the word git, a later occurrence of the token that names the rejected operation, and a bare minus-f token elsewhere in the same string. No command invoked any version-control binary, and nothing in either write could have modified any remote. That brings the measured count of benign refusals by this matcher to at least three across two sessions, all of them on evidence-file or documentation writes – which is the population most exposed to it, because documents ABOUT the guard necessarily quote the very tokens the guard matches on. WHAT THIS CHANGES ABOUT THE ASSESSMENT: nothing about the direction of failure. The guard still fails SAFE and no unsafe operation has resulted. What the recurrence establishes is that the false-positive rate on documentation work is not incidental – it is systematic and self-reinforcing, since every attempt to WRITE UP the defect is itself blocked by the defect, which is why the original bluff-hunt evidence file landed two days late. The cost remains the one already recorded: agents learn to route around a safety guard to complete ordinary work, and a guard routinely routed around has no authority left for the case it exists to catch. The closure criteria above are unchanged; the fixture set should additionally include a benign documentation string that quotes all three trigger tokens and MUST be permitted.

LVA-148 — Pipeline 002 — channel names two different axes, so any repair implementing the channel parameter would encode the collision permanently

Status: Operator-blocked **Type:** Bug **Operator-Block-Details:**

WHAT: Choosing what the two axes are called, and deciding what a release-channel evidence artifact IS, sets the vocabulary for a machine contract, a governing clause and three scripts at once, and it determines whether section 6.AA clause 8 condition (D) can be made real or must be reworded. No release-channel artifact exists to model an answer on, so the decision invents a shape rather than discovering one. Naming and evidence-shape decisions of that reach are not agent-side. **WHY:** Section 6.T.3, plus the rule that amendments to a governing clause and to this feature's machine contracts are operator decisions. Section 6.J also applies directly: implementing the channel parameter without disambiguating the vocabulary first would ship a parameter that reads as a per-variant protection and delivers a per-version one, which is the shape this feature has spent the cycle removing rather than adding. **UNBLOCK:** Operator picks one of the following **CHOICES:** [A] keep the word channel for the Firebase APP, as contracts/distribution-record.schema.json already defines it, rename the variant axis in scripts/firebase-distribute.sh and scripts/check-cycle-coverage.sh to a distinct word such as variant, and then implement that variant parameter for real; this requires first defining what release-variant evidence artifacts are, because none exist today. [B] the reverse assignment: keep channel for the BUILD VARIANT, as clause 8 condition (D) uses it, and rename the contract property to app; this is a machine-contract amendment and carries the migration cost of every consumer that reads the existing property. [C] retire the combined debug-and-release distribute mode so the second axis is never needed on that path, since the release-only mode already resolves the variant to release and already runs the staging gate, and then reword condition (D) to state what the per-version check actually verifies; this is the drafted recommendation for the related defect because it repairs the shared script rather than its description, every future caller benefits, and it preserves the no-pause requirement by running the two stages back to back, removing only the flag. [D] accept the collision and document it, in which case the channel parameter **MUST NOT** be cited anywhere as evidence that a specific variant was verified, and condition (D) is reworded to drop its two-channel language. Unblock signal: the operator records the chosen option, or a fix lands together with a fixture that distinguishes the two axes and that **FAILS** against the current inert parameter. **WHO:** Operator, or a designated reviewer with

authority over this feature's machine contracts under specs/002-build-test-distribute-pipeline/contracts/ and over CLAUDE.md section 6.AA. **Severity:** P2

WHAT: the word channel names two different axes in this feature, and the conflict sits directly under the repair path for LVA-120 and for section 6.AA clause 8 condition (D). MEASURED BY DIRECT INSPECTION: specs/002-build-test-distribute-pipeline/contracts/distribution-record.schema.json line 11 defines channel as the Firebase APP, with an enum of the client app distribution target and the api-app distribution target; scripts/firebase-distribute.sh lines 321 to 324 resolve a channel variable to release or debug, which is the BUILD VARIANT; and scripts/check-cycle-coverage.sh already receives the app-channel through its evidence-directory parameter, so its channel parameter is a SECOND axis wearing the same word. That parameter is also INERT and unvalidated: the variable is set by the two argument-parsing branches at lines 37 and 38, asserted non-empty at line 51, and never read again in the remaining 128 lines of the script, so both artifacts it checks resolve from the evidence directory and the version alone, and debug, release and an arbitrary unknown value all produce byte-identical output and exit 0. RELATED AND SEPARATELY MEASURED: no release-channel evidence artifacts exist anywhere. The evidence tree under .lava-ci-evidence/distribute-changelog partitions by APP only, into a client directory and an api-app directory, with no debug-versus-release split at any level. Implementing the channel parameter therefore requires first INVENTING what release-channel evidence IS, which is a design decision rather than a code fix. SCOPE NOTE so this is not read as a duplicate: this item is the vocabulary collision itself together with the absence of release-channel artifacts. The behavioural consequence inside firebase-distribute.sh is tracked at LVA-120, and clause 8 condition (D) inherits both. DRAFTED OPTIONS: specs/002-build-test-distribute-pipeline/amendment-defects-proposal.md sections 2 and V-5.

LVA-149 — Pipeline 002 — the orchestrator resolves git identity and evidence paths from the caller's CWD, so a run against another repository deposits a PASS report into Lava's evidence tree under Lava's own HEAD

Status: In progress **Type:** Bug **Severity:** P0 **Created-By:** AI

WHAT: scripts/pipeline-build-test-distribute.sh resolves every git query and every evidence path against the CALLER'S working directory rather than against the repository the run is supposed to

be about. Three facts compose into the defect, each verified by direct inspection of the script this session. First, the run report's commit identity is taken as `commit_sha="$(git rev-parse HEAD)"` with no `-C "$REPO_ROOT"`, so it names whatever repository the invoking shell happens to be sitting in. Second, `RUN_DIR` and `_run_report_path()` are RELATIVE paths under `.lava-ci-evidence/pipeline-runs/`, so the report is deposited relative to the caller's directory, not the repository under test. Third, the script contains ZERO `cd` statements, so nothing ever reconciles the two: it never enters `$REPO_ROOT`, and it never re-resolves the evidence root against it. `REPO_ROOT` is computed and then not used for either purpose. MEASURED, not inferred: a run was invoked against a fixture repository, and its FR-000 precondition guard passed legitimately, because the guard inspects the FIXTURE and the fixture was on master with a clean tree. The run then wrote its report into LAVA'S OWN `.lava-ci-evidence/pipeline-runs/` tree, carrying `commit_sha`
545920a160cc8e5591bb4c9c7a87793f371219de -- Lava's HEAD, a commit the run never built, never tested and never touched -- together with outcome PASS and phases [(precondition, PASS)]. IMPACT: the constitution matches evidence to artifacts BY COMMIT SHA in three separate places -- 6.Z's same-SHA test-evidence rule, 6.AK's coverage-intersection gate, and 6.AA clause 8 condition (A), which refuses a run report whose `commit_sha` does not equal the HEAD at distribute time. A report that names a SHA it never exercised is therefore not merely mislabelled: it is a false claim at rest, sitting in the exact directory those three gates read, in the exact shape they accept. This is the 6.J class -- nothing was learned about that commit, recorded as though the commit passed. CONTAINMENT ALREADY PERFORMED: three such artifacts were preserved to scratchpad forensics and removed from the evidence tree; the 57 pre-existing legitimate reports were left untouched and were not modified in any way. STATUS: a fix is IN FLIGHT with another agent in this session, which is why this item is filed In progress rather than Queued -- it is recorded here so the defect cannot evaporate if that work is interrupted (section 11.4.197). CLOSURE CRITERIA: every git invocation in the orchestrator is repository-scoped via `-C "$REPO_ROOT"` (or the script enters `$REPO_ROOT` before any of them), the evidence root is resolved as an absolute path beneath `$REPO_ROOT`, and a regression test invokes the orchestrator from a DIFFERENT working directory than the repository under test and asserts both that the report lands in the target repository's evidence tree and that its `commit_sha` equals the target repository's HEAD rather than the caller's.

LVA-151 — Parallel-agent contention in one working tree makes constitutional gates report false failures — five diagnosed this session, all dissolved on isolated re-measurement; a serialization rule or lock is owed

Status: Queued **Type:** Task **Severity:** P1 **Created-By:** AI

WHAT: multiple agents working concurrently in ONE shared working tree cause constitutional gates to report failures that are artifacts of the contention rather than properties of the tree. FIVE separate gate failures during this session each dissolved on isolated re-measurement, and no code change was required for any of them. MEASURED CASES, each re-measured in isolation before being dismissed: (a) a workable-items check reported as a timeout was slowness under concurrent load, not a hang – it completed normally once the tree was quiet; (b) a covenant-anchor check reported as a race was caused by the MEASURING process's own short timeout terminating a probe part-way through a mutation, so the measurement destroyed the state it was measuring; (c) a process-liveness check reported a stale runner that was the pgrep pattern matching the checking process itself, a self-match; (d) verify-all's markdown-export-sync gate failed because a different agent was regenerating .html and .pdf siblings during the sweep, transiently leaving a .md newer than its own exports – the isolated re-run reported 282 files examined, 0 problems, exit 0, and was repeated four consecutive times with the identical result, which makes the true verify-all figure 58 of 58 rather than the 57-with-1-failure the contended sweep produced; (e) check_constitution_test.sh's test_live_repo_passes failed twice under load and PASSED in isolation with exit 0 and 5 of 5 cases. COST: five separate diagnostic efforts were spent establishing that nothing was wrong, and in case (d) a contended sweep produced a headline number that understated the tree's real compliance – a false RED, which erodes trust in the gates in exactly the way a false GREEN erodes trust in the tests. IMPACT ON EVIDENCE: an attestation captured during contention records a verdict that is a property of the scheduling, not of the tree, so it is not reproducible and cannot serve as evidence either way. THE RULE THIS ITEM EXISTS TO RECORD, stated plainly so a sixth diagnosis is not spent rediscovering it: A CONSTITUTIONAL-GATE FAILURE IS NOT A FINDING UNTIL IT REPRODUCES IN ISOLATION. A gate result captured while other agents are writing to the tree is provisional; it is promoted to a finding only by a quiet-tree re-run, and it is that re-run's output that belongs in an evidence file. CLOSURE CRITERIA: the rule above is documented where gate-runners will read it, and either a lock file serializes gate invocations across concurrent agents or each long-running

gate acquires an advisory lock for the duration of its sweep, with a test proving two concurrent invocations serialize rather than interleave.

LVA-152 — tests/pipeline/ test_advance_all_submodules.sh — run_script assigns LAST_OUTPUT inside a command substitution, so every failure diagnostic in cases 1-5 would print a stale value

Status: Queued **Type:** Bug **Severity:** P2 **Created-By:** AI

WHAT: tests/pipeline/test_advance_all_submodules.sh defines a helper, run_script, whose body assigns the script's captured output to the GLOBAL variable LAST_OUTPUT so that failure branches can print what the script actually said. Every existing caller in cases 1 through 5 invokes that helper inside a command substitution, in the shape "\$(run_script ...)". A command substitution executes in a SUBSHELL, and a subshell's variable assignments do not propagate to the parent shell. LAST_OUTPUT in the calling shell therefore never receives the value the helper assigned: it holds whatever it held before – an empty string on the first call, and thereafter the value left by whichever earlier assignment last ran in the parent shell. CONSEQUENCE: every FAIL-branch diagnostic in cases 1 through 5 that prints LAST_OUTPUT would print a stale value rather than the output of the invocation that just failed. WHY IT HAS NEVER BEEN VISIBLE: those FAIL branches do not currently fire – the suite passes – so the stale value is never printed and nothing surfaces the defect. It is latent by construction and will surface at exactly the worst moment, when a genuine regression trips one of those branches and the operator reads a diagnostic describing a DIFFERENT invocation than the one that failed. SEVERITY RATIONALE, P2: no assertion is wrong and no result is falsified – the suite's verdicts are correct. What is wrong is the diagnostic attached to a future failure, so the cost is paid in misdirected debugging time rather than in a wrong pass. PARTIAL STATE ALREADY IN THE TREE: case 6, added during this session, does not use the helper's global at all – it captures the script's output in the CURRENT shell instead, which is the correct pattern and can serve as the model. The callers in cases 1 through 5 are unfixed. CLOSURE CRITERIA: run_script either returns its output on stdout and every caller captures it in the current shell (the case 6 pattern), or the helper is invoked without a command substitution so its global assignment reaches the caller; and one deliberately-

failing fixture asserts that the printed diagnostic contains a string unique to the failing invocation, which the current code cannot satisfy.

LVA-153 — CHANGELOG_CHANNEL still names the app axis after the LVA-148 rename, and cannot be renamed in one file alone

Status: Queued **Type:** Task **Severity:** P2

WHAT: the LVA-148 axis rename established 'app' (Firebase app: client vs api-app) and 'build_variant' (release vs debug) as the two distinct names. CHANGELOG_CHANNEL was deliberately NOT renamed, because scripts/pipeline/phase-05a-changelog-entry.sh:306 greps scripts/firebase-distribute.sh for the LITERAL string CHANGELOG_CHANNEL="", and two tests/pipeline/ files mirror that literal. Renaming it in firebase-distribute.sh alone would break a live gate at a distance, which is why the rename agent stopped rather than proceeding. OWED: one ATOMIC rename CHANGELOG_CHANNEL -> CHANGELOG_APP across 4 files / 5 sites, landed together. CONTAINED MEANWHILE: test case P5 asserts the variable can only ever hold an app slug, so it cannot silently acquire a build-variant value while the rename is pending. Source: self-discovered during the LVA-148 rename, 2026-08-26.

LVA-154 — Governance prose and data-model still call the build-variant axis a channel after the LVA-148 rename

Status: Queued **Type:** Task **Severity:** P2

WHAT: the LVA-148 rename separated 'app' from 'build_variant' in the contracts and scripts, but the prose that describes them was not updated in the same pass. Affected: CLAUDE.md 6.AA clause 8 still calls the build-variant axis a channel; specs/002-build-test-distribute-pipeline/spec.md likewise; and specs/002-build-test-distribute-pipeline/data-model.md's table now DESCRIBES A PROPERTY THAT NO LONGER EXISTS, since distribution-record.schema.json's channel was renamed outright with no alias (0 records at rest, so nothing became unreadable). Also noted at the same time: scripts/tag.sh:285-306 uses channel_label / channel_dir for a THIRD sense of the word, and scripts/pipeline/phase-02-test-hermetic.sh:72-75 cites tests/firebase/repro_mode_both_channel_gap.sh, which was deleted earlier the same day when combined distribute mode was retired. WHY IT

MATTERS: documentation describing a property that does not exist sends the next reader to look for it, and the whole point of the rename was that one word naming two axes is a trap. Governance edits require operator review, which is why this is filed rather than done. Source: self-discovered during the LVA-148 rename, 2026-08-26.

LVA-155 — Pre-push Check 10 passes the pointer-advancing commit as -head, which cannot contain its own SHA, so the next distribute-pointer commit will be rejected at push time

Status: Queued **Type:** Bug **Severity:** P1

WHAT: .githooks/pre-push Check 10 invokes scripts/check-cycle-coverage.sh with -head=\$sha where \$sha IS the commit advancing the distribute pointer. The evidence that commit references was written BEFORE it existed, so it cannot declare that commit's own SHA. Now that the LVA-149/F2 fix makes the commit-SHA binding actually bind (it was inert, which is why 5 of 9 shipped cycles passed against an arbitrary wrong HEAD), this self-reference surfaces: the next pointer-advancing commit will be REJECTED at push time. MEASURED: the binding fix took wrong-HEAD acceptance from 5/9 to 0/9; before the fix this rejection was hidden. NOT closed by the fixing agent, deliberately and correctly: it declined to implement an ancestor/code-identity escape because it could not prove the acceptance path with a hermetic positive case, and an untested acceptance path inside a safety gate is precisely the defect class it was dispatched to remove. Three candidate options are written up in the evidence log. DOES NOT BLOCK the current push: Check 10 fires only on commits that advance a distribute pointer, and the pending change set touches no last-version or distribute-changelog path (verified). Source: self-discovered during the LVA-149 fix, 2026-08-26. Needs an operator decision before the next distribute.

LVA-156 — Cycle 1.3.12-1077 asserts 13 covering Challenge refs while its evidence records verdicts for two — the 6.AK incident shape recurring one cycle after 6.AK was written

Status: Queued **Type:** Bug **Severity:** P1

WHAT: re-running the REPAIRED cycle-coverage gate over all 9 shipped cycles shows 8 hold up on substance — the gate was broken, but those releases were not shipped on nothing. 1.3.12-1077 is the exception and it FAILS under the fixed gate: 2 of 3 claims uncovered, and 11 of the 13 covering refs its cycle-coverage-map asserts have NO verdict record in the evidence at all. One asserted ref is the prose 'Full Challenge suite proven on Genymotion' standing in for 9 named Challenges. This is exactly the shape of the 2026-06-26 incident (.lava-ci-evidence/sixth-law-incidents/2026-06-26-c00-only-gate-shipped-broken-flows.json) that 6.AK was written to prevent, recurring ONE CYCLE after the clause landed — because the gate that was supposed to catch it could not read the evidence format the cycle used. Two further cycles, 1.3.17-1084 and 1.3.17-1085, declare NO commit SHA at all in their evidence. NOT a live regression: it is a historical finding about what was actually verified versus what was reported. Recorded so the record of that release is honest. Source: self-discovered during the LVA-149 blast-radius assessment, 2026-08-26.

LVA-157 — Section 6.AA clause 8's own qualifying test fixture could satisfy condition (B) with Evidence Records nobody ever validated, all of them Challenge00CrashSurvival

Status: Queued **Type:** Bug **Severity:** P0

WHAT: the three suites asserting 6.AA clause 8 (test_phase_05_distribute_gate.sh, _vacuity.sh, _fix_audit.sh) each call write_evidence_record once and validate_evidence_record ZERO times. Until 2026-08-26, write_evidence_record stamped every record with anti_bluff_status 'validated' — the SAME value the independent validator writes on accept — so a record nobody examined was byte-identical on disk to one examined and accepted, and every consumer read that field as a verdict. The aggregator compounded it by treating an absent status as 'not rejected', an asymmetry against 'result', where an unrecognised value already fell through to FAIL. NET EFFECT: clause 8 condition (B) — 'zero Evidence Records carry an anti_bluff_status other than validated' — was satisfiable by records that were never validated, INCLUDING all four mandatory real-device-challenge records for app-debug, app-release, api-app-debug and api-app-release, every one of them Challenge00CrashSurvival. Clause 8 is the rule authorising a distribution to proceed from debug to release WITHOUT an operator pause. WHY THIS IS P0: that is the 2026-06-26 'C00-only gate shipped broken flows' forensic anchor (.lava-ci-evidence/sixth-law-incidents/) reproduced ONE LEVEL

ABOVE where the sweep found it — not in a release's evidence, but in the test fixture that CERTIFIES the gate authorising unattended release. 6.Z clause 4 states a cold-start check is the minimum and explicitly NOT sufficient. ATTRIBUTION SETTLED by controlled A/B, not inferred: swapping only evidence.sh + phase-02-test.sh gives pristine -> all three EXIT=0, fixed -> all three EXIT=1; the suites were relying on the fail-open default. UNDERLYING FIX LANDED: the writer now emits a schema-legal fail-closed placeholder and the aggregator classifies three ways (validated / REJECTED* / anything else = UNEVALUATED, which fails the phase). OPEN: the fixtures need repair, and it is not yet established whether clause 8 can still qualify on C00-only evidence once they validate properly — that is being determined now. Source: self-discovered during the tag.sh critical fail-open fixes, 2026-08-26.

LVA-158 — The FR-000 clean-tree precondition is blind to untracked leftovers inside submodules, because .gitmodules sets ignore = untracked

Status: Queued **Type:** Bug **Severity:** P1

WHAT: .gitmodules declares 'ignore = untracked' for submodules, so 'git status' in the parent does not report untracked files left inside a submodule working tree. The pipeline's FR-000 precondition guard reads that status to decide the tree is clean, and therefore PASSES with untracked leftovers present in two submodules. STATUS CHANGE: the 2026-08-26 vacuous-pass sweep recorded this as P11 UNCONFIRMED; it is now CONFIRMED BY FIXTURE during the corpus-floor sweep. WHY IT MATTERS: FR-000 exists so the pipeline never builds, tests or distributes from a tree carrying uncommitted state — a run that verifies artifacts built from unrecorded local changes attributes its evidence to a commit that does not describe what was actually built. That is the same attribution failure class as LVA-149 (a run report claiming a commit_sha the run never tested), reached by a different route. Composes with the T054 re-review finding that every push and both B-2 residuals in advance-all-submodules.sh sit behind has_local_mods — a precondition the script neither checks nor states. Source: self-discovered, sweep finding P11 promoted from UNCONFIRMED to CONFIRMED, 2026-08-26.

LVA-159 — tests/check-constitution/ test_credential_scan_corpus.sh has been asserting nothing — its git stub ignores -z, so the section 6.R scanners scanned zero files under test

Status: Queued **Type:** Bug **Severity:** P1

WHAT: the test's 'git' stub does not honour the -z flag, so the file list it feeds the three section 6.R scanners (scan-no-hardcoded-uuid.sh, scan-no-hardcoded-ipv4.sh, scan-no-hardcoded-hostport.sh) is not parsed as the scanners expect, and they examine ZERO files. The suite has therefore been green while asserting nothing about credential or hardcoded-literal detection. PROVED BY HEXDUMP during the 2026-08-26 corpus-floor sweep; it became visible only because the new corpus floor made a zero-file scan refuse instead of silently succeeding. SCOPE, stated precisely: the defect measured is in the TEST'S STUB, so what is established is that the TEST was vacuous — NOT that the production scanners are broken against a real corpus. Confirming production behaviour is separate work and is NOT claimed here. WHY IT MATTERS: section 6.H treats credential leakage as a security incident and section 6.R forbids hardcoded literals; a suite that certifies those scanners while feeding them nothing is the highest-consequence instance of the vacuous-pass family found this cycle. NOT FIXED: the file is staged by another agent's concurrent work; the remedy is one line in the stub. Source: self-discovered during the corpus-floor sweep, 2026-08-26.

LVA-160 — Section 6.AA clause 8 condition (C) never reads WHICH Challenge ran, so the clause can authorise an unattended debug-to- release distribute on cold-start-only evidence

Status: Queued **Type:** Bug **Severity:** P0

WHAT, MEASURED not argued (CASE 19 probe in tests/pipeline/test_phase_05_distribute_gate.sh, re-measured every run rather than asserted in prose): with 'records naming a Challenge OTHER than Challenge00CrashSurvival: 0', the gate reports 'CONDITION (C) Device evidence for BOTH variants : PASS' and 'clause 8 QUALIFIES (exit 3)'. Condition (C) at scripts/pipeline/phase-05-distribute.sh:908-1022 checks variant binding, category, result,

anti_bluff_status and the container-or-VM runner — it NEVER reads the test_id. WHY THIS IS P0: clause 8 is the rule authorising a distribution to proceed from debug to release WITHOUT an operator pause, and clause 8(C) states IN ITS OWN TEXT that cold-start ‘does not by itself satisfy 6.AK’. The implementation does not enforce what the text asserts. This is the 2026-06-26 ‘C00-only gate shipped broken flows’ incident condition, reachable through the gate written to prevent it. AGGRAVATED BY THE 2026-08-26 AMENDMENT: the re-lettering withdrew former condition (D) — the only condition gesturing at per-variant channel verification — recording it as 6.AA-pipeline-debt-D OWED rather than replacing it, on an analysis stating that ‘(C) becomes MORE load-bearing, not less: with (D) retired it is the sole mechanical release-variant guarantee’. That was true of the STRUCTURE and false of the SUBSTANCE, and the assumption was not tested before the amendment was applied. THE FIX (specified, NOT implemented — the gate was deliberately not touched): require, per variant, at least one PASSing real-device-challenge record whose test_id is NOT the cold-start canary — equivalently, intersect the executed-PASS Challenge set with the cycle’s claim-set per 6.AK clause 1. Nothing implements this today. Source: self-discovered during the clause-8 fixture repair, 2026-08-26.

LVA-161 — phase-02-test-challenge.sh runs 73 Challenge classes in ONE emulator-matrix invocation with no -test-timeout, so the default 10-minute kill marks 73 classes FAIL and reads as a broken product

Status: Queued **Type:** Bug **Severity:** P1

WHAT, measured live during the T062 end-to-end run (run_id 2026-08-26T14-09-17Z): the test phase reported 74 real-device-challenge FAILs. They are a TIMEOUT ARTEFACT, not 74 broken features. phase-02-test-challenge.sh selects 73 Challenge classes in a SINGLE emulator-matrix invocation, and NEITHER it NOR scripts/run-challenge-matrix.sh passes -test-timeout (verified: zero occurrences in either file), so cmd/emulator-matrix’s DEFAULT 10 minutes applied. Attestation row: test_seconds=600.02, test_error=‘signal: killed’. Gradle’s last log line: ‘Tests 81/104 completed. (6 skipped) (12 failed)’. Gradle was killed before writing JUnit XML, so the phase parsed 0 files and marked all 73 classes FAIL. Of what actually ran, 12 genuinely failed (e.g. Challenge46SearchTimeoutInterruptTest -> ComposeTimeoutException after 15000 ms). UNCONFIRMED: the true status of the remaining ~61 — settled by raising -test-

timeout or batching the classes. The api-app half is a SEPARATE, GENUINE failure (test_seconds=124.88, exit status 1), not a timeout. WHY IT MATTERS beyond the wasted run: a future operator reading this evidence would conclude the product is broken when it is not, and the same reasoning error in reverse is how a real breakage gets dismissed as 'just the timeout'. WHAT THE PIPELINE GOT RIGHT, worth preserving: it fabricated no passes for the killed classes, wording them 'No entry found ... 0 file(s) parsed — either the class filter did not match, or the run aborted before instrumentation completed'. Source: self-discovered during the T062 run, 2026-08-26.

LVA-162 — The gating emulator attestation row carries an empty diag, so tag.sh Group-B Gate 3 has nothing to compare and the AVD-shadow bluff check is inert

Status: Queued **Type:** Bug **Severity:** P1

WHAT, measured during the T062 end-to-end run: the per-AVD attestation row is marked gating: true but carries diag = {} and failure_summaries = []. Section 6.I.4 (Group B extension) requires every row to carry diag.target / diag.sdk / diag.device / diag.adb_devices_state — the per-AVD forensic snapshot captured immediately before instrumentation — plus parsed failure_summaries. scripts/tag.sh Group-B Gate 3 exists to catch the AVD-SHADOW BLUFF by comparing diag.sdk against api_level and refusing when they differ; with diag empty, that comparison has nothing to read. The gate is therefore inert on exactly the rows it is meant to police, and it is inert on a row that declares itself gating. This is the vacuous-pass family (a gate reaching a verdict having examined zero items) landing inside the emulator attestation path, which the 2026-08-26 sweep did not cover because it swept scripts rather than emitted evidence. Composes with LVA-160 (clause 8 condition (C) never reads WHICH Challenge ran): two independent device-evidence gates, each unable to see the field it exists to check. Source: self-discovered during the T062 run, 2026-08-26.

LVA-163 — Recursive submodule scope contains 28 third-party remotes; a literal recursive push would attempt to write to repositories we do not own

Status: Queued **Type:** Task **Severity:** P1

WHAT, measured 2026-08-26 after initialising the previously-uninitialised nested submodules at their already-recorded pins (zero pins moved): the recursive submodule set carries 69 remotes, of which 41 are own-org (vasic-digital / HelixDevelopment) and 28 are THIRD-PARTY. They include github.com/google/perfetto, bytedance/UI-TARS and UI-TARS-desktop, square/leakcanary, appium/appium, Genymobile/scrcpy, chroma-core/chroma, allure-framework/allure2, anthropics/anthropic-quickstarts, SigNoz/signoz, Skyvern-AI/skyvern, run-llama/llama_index, mem0ai/mem0, kiwitcms/Kiwi and 15 more - nearly all vendored under submodules/helixqa/tools/opensource/ - PLUS THE TOP-LEVEL SUBMODULE NAMED superspec, which points at github.com/WangX0111/superspec. IMPACT: the operator directive 'commit and push all Submodules fully recursively to all upstreams', executed literally, would attempt a push to 28 repositories owned by other people and organisations. Such attempts would almost certainly fail for lack of write access, but attempting them is wrong, and a credential that DID grant access would make it an incident rather than a failed command. CONVERGING MEASUREMENT from the T055 design pass: the 27 previously-uninitialised submodules are EXACTLY submodules/helixqa/tools/**, and every one has a third-party upstream - so they are precisely the 27 that root CLAUDE.md's Automated Pipeline Pin-Advance Path condition (C) forbids advancing. Initialising them makes them RECORDABLE, never ADVANCEABLE. NOTE the governance default-deny introduced by LVA-138 is what contains this today: superspec was one of the seven submodules named by NO operator decision, and it points at a third-party repository - the allow-list defaulting to deny is doing real work. OPERATOR DECISION 2026-08-26: push own-org remotes ONLY, and REFUSE each third-party remote with a named per-remote record rather than skipping it silently - 'all submodules handled' without per-submodule records is not evidence, by the same reasoning section 6.I applies to per-AVD attestation rows. Aligns with section 6.W (which governs where WE push) and with condition (C). FILING NOTE: the first version of this record was mangled by unescaped backticks in the authoring shell command, which silently dropped the submodule name in two places. Corrected here; recorded because a tracker entry that quietly lost a fact is the same defect class this cycle has been evicting all day.

LVA-164 — Initialising the nested submodules left helixqa permanently dirty via an upstream CRLF artifact, making the ONLY allow-listed submodule un-advanceable and failing FR-000

Status: Queued **Type:** Bug **Severity:** P1

WHAT, measured 2026-08-26 immediately after initialising the 29 nested submodules at their already-recorded pins: submodules/helixqa now reports a permanent dirty state at the parent level (M submodules/helixqa), caused by a line-ending artifact three levels down. CHAIN, traced end to end: helixqa -> tools/opensource/docling (gitlink pin 4f37eeee == actual HEAD 4f37eeee, so NO pin moved) -> tests/data/uspto/sources/pftaps057006474.txt. That file ships with MIXED line endings (825 CRLF among 1380 lines, 75716 bytes) while docling's own .gitattributes declares '* text=auto', so git normalises on checkout and the worktree can never match the index. Git states the mechanism itself: 'CRLF will be replaced by LF the next time Git touches it'. DECISIVE TEST: stripping CR from both the index blob and the worktree file yields byte-identical content, so the difference is line endings ONLY - there is no content change and no pin move. NOTE on instruments: 'git diff -ignore-cr-at-eol' does NOT neutralise this and reports 1 differing file, because the endings are mixed rather than uniformly CRLF-at-EOL. An earlier check using that flag as the discriminator reached the wrong conclusion; the correct discriminator is a CR-stripped byte comparison. Recorded because using the wrong instrument and believing its answer is the defect class this cycle has been evicting. WHY THIS BLOCKS: submodules/helixqa is the ONLY submodule the governance allow-list permits advancing (LVA-138 inverted the list to default-deny with helixqa as the sole entry), and the T054 round-2 fix made an unclean submodule working tree a REFUSAL by default, opt-in only via a flag the pipeline never passes. So the one advanceable submodule is now un-advanceable. It also fails the pipeline's FR-000 clean-tree precondition at the parent level. Neither was true before the init. REPRODUCIBILITY: inherent to docling@4f37eeee and its .gitattributes; it reproduces on any fresh clone on a platform where git normalises. It is not local damage and must not be 'fixed' by committing the normalised file into a third-party submodule we do not own. CANDIDATE REMEDIES, none yet chosen: (a) set core.autocrlf=false / core.eol=lf for that submodule and re-check-out so the worktree matches the index; (b) add a local .git/info/attributes marking that path binary so git stops normalising; (c) teach the clean-tree precondition to treat an eol-only difference as clean - REJECTED as a default because it

weakens the very check that makes the advance safe, and would need to be provably scoped to eol-only; (d) do not initialise docling at all, which conflicts with the operator's all-59-recursive decision. Source: self-discovered while verifying the consequences of the recursive init, 2026-08-26.

LVA-165 — GitLab token printed to the session transcript by a colon-dash default expansion - the identical mistake LVA-5 records with the Firebase token

Status: Queued **Type:** Bug **Severity:** P0

WHAT: during the T054 architectural analysis an agent wrote a shell expansion of the form DOLLAR-BRACE GITLAB_TOKEN COLON-DASH NO BRACE, intending a presence check. That operator expands to the variable's VALUE when the variable is set, so a real GitLab personal-access token was printed into the session transcript. CONTAINMENT, verified by measurement not assertion: 0 tracked files match a token pattern (git ls-files piped to grep), and 0 scratchpad artefacts contain an unredacted token. The token did NOT reach git. THIS IS A REPEAT. CLAUDE.md's 67th section-6.L cycle records the IDENTICAL mistake with LAVA_FIREBASE_TOKEN, filed as LVA-5, where the operator chose to accept the risk rather than rotate. The same footgun has now fired twice in one codebase, in two different agents, months apart. That is the signal worth acting on: it is not an individual lapse, it is a shell idiom that reads like a safe presence-check and is not one. OPERATOR ACTION REQUIRED per section 6.H clause 6: rotate the GitLab token. REMEDY WORTH CONSIDERING beyond rotation: a presence check must never use the COLON-DASH default operator. The safe forms are a test on the variable being non-empty, or printing only a fixed literal such as SET or UNSET. A grep gate over tracked scripts and agent-authored commands for COLON-DASH applied to any name matching TOKEN, SECRET, KEY, PASSWORD or CREDENTIAL would catch the class mechanically rather than relying on nobody making the mistake a third time. Source: self-discovered and self-reported by the acting agent, 2026-08-26.

LVA-166 — The script's stated clean-tree safety property is false - the verify step runs unguarded on the flagless path with live push credentials, invalidating the basis of two review approvals

Status: Queued **Type:** Bug **Severity:** P0

WHAT, proven against the current file: the safe-operating precondition stated in the script's own header at lines 116-125 says that with a CLEAN submodule tree the run is 'all local, all undoable, nothing leaving this machine'. That is FALSE. PUBLISH_LOCAL_MODS is last consulted at line 1810; the step-5 verify command is invoked later and is NOT gated by it, so the verify step executes on the DEFAULT, FLAGLESS, CLEAN-TREE path with the operator's live credentials in reach. The analysis agent demonstrated the capability by performing an SSH-authenticated git push from an ordinary child process (target was a local bare repo over ssh-localhost; no real upstream was contacted). The operator's SSH key is unencrypted on disk. 25 own-org upstreams are reachable. WHY THIS MATTERS MORE THAN ANY SINGLE ESCAPE: review rounds 4 and 5 both APPROVED the script conditional on not passing -publish-local-modifications, on the stated basis that the clean-tree path creates no commit and issues no push and had 'survived two full adversarial passes'. That conclusion rests on a safety property the code does not implement. Five review rounds and eleven proven escapes were all reasoned about within a boundary that does not exist. ADDITIONAL MEASURED FACTS from the same analysis: (1) environment scrubbing does NOT close this - 'env -i' with HOME redirected to an empty directory STILL pushed successfully, because OpenSSH resolves the tilde from the passwd entry rather than from HOME; (2) path masking DOES work - bwrap with HOME and /run on tmpfs made both push routes fail while leaving network intact; (3) isolation closes ZERO of the eleven escapes, proven rather than argued, because a fully sandboxed verify step still poisoned the commit headers - the SCRIPT publishes afterwards; what isolation removes is review 5's strongest argument for the futility of guarding. RESIDUAL, stated honestly: a sandbox still has write access to the tree it builds, so it can lie about the build; cache poisoning across the boundary is real and unaddressed since the gradle cache must be writable and a later unsandboxed build executes what it finds. UNCONFIRMED: whether a real Lava build completes inside the sandbox - that experiment gates the whole item. Source: self-discovered during the T054 architectural analysis, 2026-08-26.

LVA-167 — The verify-step sandbox withholds git credentials but not the in-tree secrets the build is given - a sandboxed build still uploaded to Firebase twice

Status: Queued **Type:** Bug **Severity:** P1

WHAT: the credential-masking sandbox prototyped for LVA-166 is a GIT-CREDENTIAL sandbox, not a credential sandbox, and the difference is load-bearing. Masking HOME and /run/user removes the SSH agent socket and the on-disk key, which provably blocks git push (probe fired 45s into a live 17-minute build: both routes exit 128, zero refs; falsified by an unsandboxed control that succeeds, and by removing only the two tmpfs lines, after which both pushes land). But the build must be GIVEN the repository tree, and that tree carries live secrets: .env (4072 bytes - Firebase token, HMAC secret, tracker credentials), app/google-services.json and api-app/google-services.json, and keystores/release.keystore plus keystores/debug.keystore. All are gitignored, so section 6.H holds with respect to git - but gitignore does not withhold them from the build. PROVEN, not argued: the SANDBOXED build ran uploadCrashlyticsMappingFileRelease TWICE, i.e. it reached Firebase from inside the sandbox. So the claim 'nothing leaving this machine' remains FALSE even with credential masking in place. Whoever corrects the false header recorded in LVA-166 must not substitute a smaller falsehood.

CONTEXT - the sandbox itself WORKS and this is not an argument against it: a real 4-variant Lava Android build completed inside it (BUILD SUCCESSFUL in 17m 50s, exit 0, all four APKs apksigner-verified with real Lava certs, 3-ABI Go+cgo liblavaapi.so cross-compiled through the NDK), a cold-cache rebuild was byte-identical to the host's, and cache reuse survives (2682/3035 tasks UP-TO-DATE, ~54ms fixed entry cost, A/B wall-time deltas of +42/+46/-42s on a ~130s build - sign flips, indistinguishable from noise). TWO FURTHER RESULTS worth keeping: (a) the architecture analysis's own prototype masks /run WHOLESale, but this repository lives at /run/media/... - following it literally deletes the project and fails in 0.01s with 'cd: No such file or directory'. An implementer following it would have recorded a FALSE NEGATIVE and discarded a working control. Mask /run/user, never /run. (b) the cache-poisoning residual the analysis disclosed and could not close IS closable: -ro-bind on the gradle cache fails (the Gradle WRAPPER needs a writable .lck and the build dies at 0s), but bwrap -overlay-src with -tmp-overlay works - build exit 0, identical work, host cache verifiably unpoisoned, push still blocked, at roughly 3x warm wall time. The poisoning was confirmed end-to-end first: a sandboxed process writes gradle init.d/ and a later UNSANDBOXED Gradle executes it with the

agent socket present. NOT ESTABLISHED: only the BUILD half was measured. phase-02-test.sh under masking is UNCONFIRMED and is the higher-risk half, because section 6.AH emulators interact with the mount namespace. Source: self-discovered during the LVA-166 sandbox feasibility experiment, 2026-08-27.

LVA-168 — A skipped phase is omitted from the run report entirely, so a zero-test run reports outcome PASS with every phases[] entry PASS

Status: Queued **Type:** Bug **Severity:** P0

WHAT, measured live during T062 attempt #4: the pipeline run report never discloses a SKIPPED phase. A run invoked with `-skip test` omits 'test' from `phases[]` ENTIRELY, so the report shows outcome PASS with every listed entry PASS - a run that executed ZERO tests is byte-indistinguishable in its report from one that ran the full suite and passed. Captured: run 2026-08-27T00-15-42Z, exit 0, precondition PASS / build PASS / test SKIPPED / install_boot PASS / live_verify PASS, with no 'test' entry in `phases[]` at all. WHY THIS IS SERIOUS: section 6.AA clause 8 condition (B) requires that every entry in `phases[]` is PASS and that zero Evidence Records carry a non-validated `anti_bluff_status`. A run that SKIPPED the phase which produces nearly all Evidence Records satisfies (B) vacuously - the array is all PASS because the phase that could have failed is not in it. This is the vacuous-pass family, at the run-report level, inside the artifact that clause 8 reads to authorise an unattended debug-to-release distribute. It composes with LVA-160 (condition (C) never reads WHICH Challenge ran) and LVA-157 (clause 8's own certifying fixtures never validated their records): three independent ways the same clause can qualify on evidence that proves nothing. THE FIX: a skipped phase MUST appear in `phases[]` with an explicit SKIPPED result, and clause 8's evaluation must treat a SKIPPED phase as disqualifying rather than absent. An omitted phase and a passed phase must never be the same bytes. SELF-CORRECTION recorded by the measuring agent: it had earlier asserted that a `-skip run` 'can never report outcome PASS', taken from a code comment rather than from measurement. The measurement disproved the comment. Recorded because trusting a comment over an experiment is the defect class this cycle has been evicting. Source: self-discovered during the T062 end-to-end run, 2026-08-27.

LVA-169 — The test phase litters untracked un-gitignored evidence dirs inside lava-api-go, so FR-000 refuses every subsequent run in that checkout

Status: Queued **Type:** Bug **Severity:** P1

WHAT, measured during T062 attempt #4: scripts/pipeline/phase-02-test.sh writes lava-api-go/internal/*.lava-ci-evidence/ directories that are neither tracked nor gitignored. The pipeline's own FR-000 precondition requires a clean working tree, so once the test phase has run, EVERY subsequent run in that checkout is refused - the pipeline dirties the tree it is about to test and then cannot start again. This is the identical shape as the pipeline-runs/ignore-rule defect closed earlier this cycle by a preventive check-ignore guard, reached by a different path: that guard probes .lava-ci-evidence/pipeline-runs/ specifically and does not see these nested per-package directories. Source: self-discovered during the T062 end-to-end run, 2026-08-27.

LVA-170 — An empty diag is refused by phase-02 but accepted by phase-04, from the same frozen emitter

Status: Queued **Type:** Bug **Severity:** P2

WHAT, measured during T062 attempt #4: the LVA-162 fix added a Lava-side refusal for a gating attestation row carrying an empty diag, and it correctly FIRED in phase-02 on live data. phase-04 accepts the same empty-diag row from the same frozen submodule emitter (submodules/containers pkg/emulator matrix.go writeAttestation). So one phase enforces section 6.I.4's diag requirement and another does not, on identical input. A gate that is enforced on one path and not another is worse than one enforced nowhere, because its presence implies coverage it does not provide. Source: self-discovered during the T062 end-to-end run, 2026-08-27.